

UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

**FACULTAD DE INGENIERÍA EN CIENCIAS DE LA
COMPUTACIÓN Y TELECOMUNICACIONES**



**Examen Parcial 2
Software de Incidencias en la UAGRM**

Estudiante: Choque Calle Jorge

Registro: 219074240

Materia: Arquitectura de Software

Sigla: INF 511

Docente: Ing. Josue Obed Veizaga Gonzales

Semestre 1/2026

Santa Cruz De la Sierra – Bolivia

Contenido

Contenido

CAPITULO 1: REQUISISTOS	3
1.1. Requisitos Funcionales	3
1.2. Requisitos No Funcionales.....	4
1.3. Identificar Actores	4
1.4. Identificar Casos de Uso	6
1.5. Priorizar Casos de Uso	7
1.6. Detallar Casos de Uso	8
CU-04:Gestion de personal	9
CU-06:Gestion del estado de Incidencia	1
1.7. Diagrama general de Casos de Uso	5
CAPITULO 2: ANALISIS	6
2.1. identificación de Paquetes	6
2.2. relacionar paquetes con Casos de Uso.....	7
CAPTULO 3: DISEÑO	2
3.1. Diseño de la Arquitectura	2
3.2. Diseño de la Base de Datos	3
3.2.1. Diseño conceptual	3
3.2.2. Diseño Logico	7
3.2.3. Diseño Fisico.....	10
3.3. Diseño de detalle procedimental.....	4
CU-04:Gestion de usuarios	5
CU-06:registro de notas de problemas.....	9
Código	14

CAPITULO 1: REQUISISTOS

El software Permitra que la comunidad universitaria (estudiantes, docentes y administrativos) reporte anomalías en servicios críticos como climatización, redes eléctricas, fontanería y mobiliario, asegurando que cada reporte sea canalizado a las autoridades y técnicos correspondientes para su pronta resolución.

1.1. Requisitos Funcionales

RF - 01: Administración de Informantes

El sistema deberá permitir el registro y la validación de los datos de contacto de cualquier miembro de la comunidad que identifique una anomalía en el campus.

RF - 02: Ciclo de Vida de Tickets de Incidencia

La plataforma debe garantizar la gestión integral (creación, edición, consulta y cierre) de las solicitudes de reparación, manteniendo la integridad de la información en cada etapa.

RF - 03: Georreferenciación de Infraestructura

Se requiere un desglose jerárquico para la ubicación de las fallas, permitiendo especificar la Facultad, el Bloque/Módulo, el Nivel y el número de Aula u oficina afectada.

RF - 04: Control de Especialistas de Mantenimiento

El software permitirá la administración del personal técnico especializado, facilitando su alta, baja y actualización de datos para la asignación de tareas.

RF - 05: Repositorio de Evidencia Visual

Se debe integrar la capacidad de adjuntar archivos multimedia (fotografías o capturas) que sirvan como prueba documental del estado de la incidencia antes y después de la intervención.

RF - 06: Módulo de Supervisión de Autoridades (Decanos)

El sistema permitirá gestionar a los Decanos de cada facultad, quienes actuarán como nodos de supervisión para validar reportes y delegar las órdenes de trabajo al personal técnico.

RF - 07: Trazabilidad y Auditoría de Estados

Debe existir un panel de consulta que permita visualizar el flujo histórico de cada incidencia, detallando quién la reportó, quién la atendió, los cambios de estado y las pruebas de resolución.

RF - 08: Sistema de Privilegios por Roles (RBAC)

El acceso a las funciones del sistema estará restringido mediante un modelo de control de acceso basado en roles, diferenciando entre Reportador, Técnico de Mantenimiento y Administrador/Autoridad.

RF - 09: Catálogo de Unidades Académicas

El sistema debe proveer herramientas para la gestión (CRUD) de la información de las Facultades que integran la universidad.

1.2. Requisitos No Funcionales

RNF - 01: Optimización de la Experiencia de Usuario (UX)

La interfaz debe ser minimalista y fluida, reduciendo al mínimo la cantidad de clics necesarios para completar un reporte de incidencia.

RNF - 02: Eficiencia de Procesamiento

El software debe garantizar tiempos de respuesta inferiores a 2 segundos en la persistencia y consulta de datos bajo condiciones normales de carga.

RNF - 03: Integridad y Seguridad de Datos

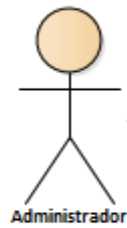
Es obligatorio el uso de protocolos de autenticación robustos (JWT) para asegurar que solo usuarios autorizados puedan interactuar con el backend, protegiendo la información contra accesos no permitidos.

RNF - 04: Adaptabilidad Multiplataforma (Responsive Design)

Dado el alto uso de dispositivos móviles, la interfaz debe ser 100% responsiva, adaptándose automáticamente a diferentes resoluciones de pantalla sin perder funcionalidad.

1.3. Identificar Actores

uc Administrador

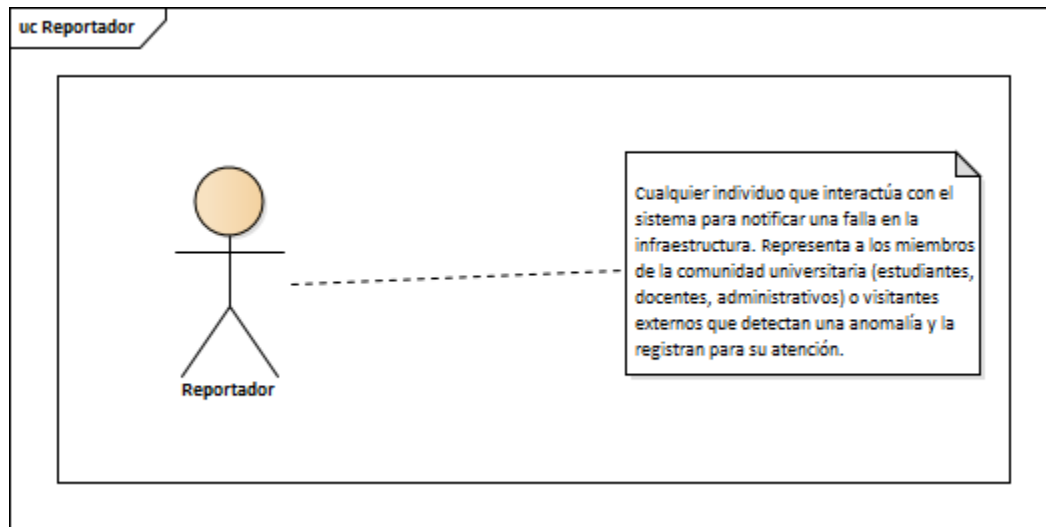


Actor encargado de la gestión integral del sistema. Sus responsabilidades incluyen la administración de cuentas de usuario, la gestión de facultades y módulos, la asignación de las incidencias reportadas al personal técnico y la supervisión del historial de atencione

uc Personal de mantenimiento



Actor responsable de atender y solucionar las fallas físicas dentro de la facultad. Recibe las incidencias que le son asignadas, acude a la ubicación señalada para realizar las reparaciones necesarias y se encarga de registrar en el sistema la finalización del mantenimiento.



1.4. Identificar Casos de Uso

CU - 01: iniciar de sesión

CU - 02: Cerrar de sesión

CU - 03: Registro de los reportadores de incidencia

CU - 04: Gestión de Personal

CU - 05: Gestionar infraestructura física

CU - 06: Gestion del estado de incidencia

CU - 07: Gestionar facultades

CU - 08: Registro de prueba de evidencia

CU - 09: Asignación de personal a incidencia

CU - 10: Visualizar incidencia

CU - 11: Finalizar orden de mantenimiento

CU - 12: Consulta de historial de incidencia

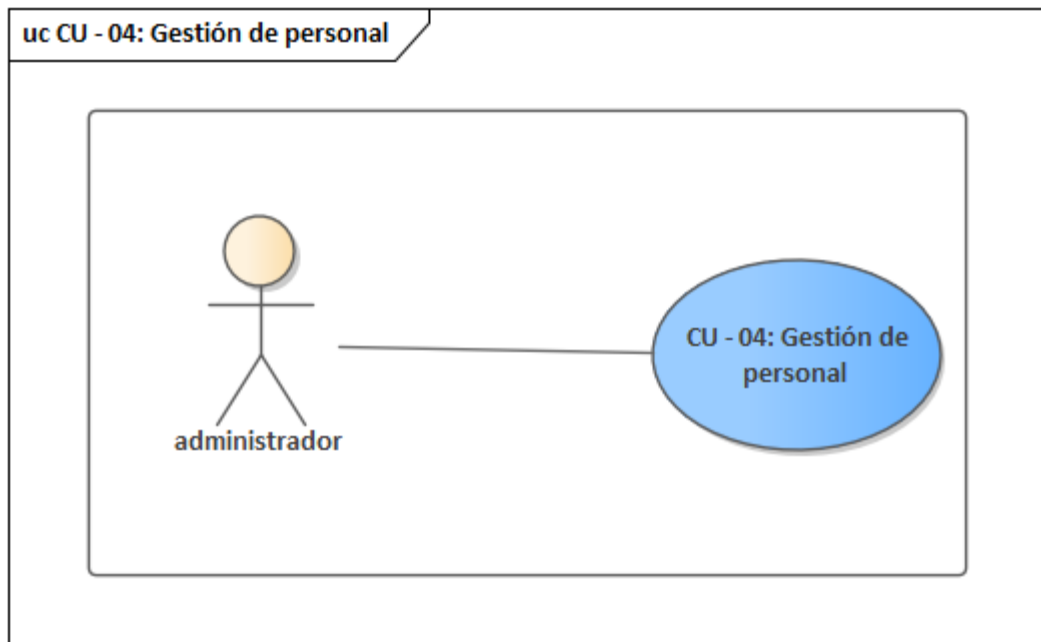
1.5. Priorizar Casos de Uso

ID	Nombre del Caso de Uso
CU - 01	iniciar de sesión
CU - 02	Cerrar de sesión
CU - 03	Registro de los reportadores de incidencia
CU - 04	Gestión de personal
CU - 05	Gestionar infraestructura física
CU - 06	Gestion del estado de incidencia
CU - 07	Gestionar facultades
CU - 08	Registro de prueba de evidencia
CU - 09	Asignación de personal a incidencia
CU - 10	Visualizar incidencia

ID	Nombre del Caso de Uso
CU - 11	Finalizar orden de mantenimiento
CU - 12	Consulta de historial de incidencia

1.6. Detallar Casos de Uso

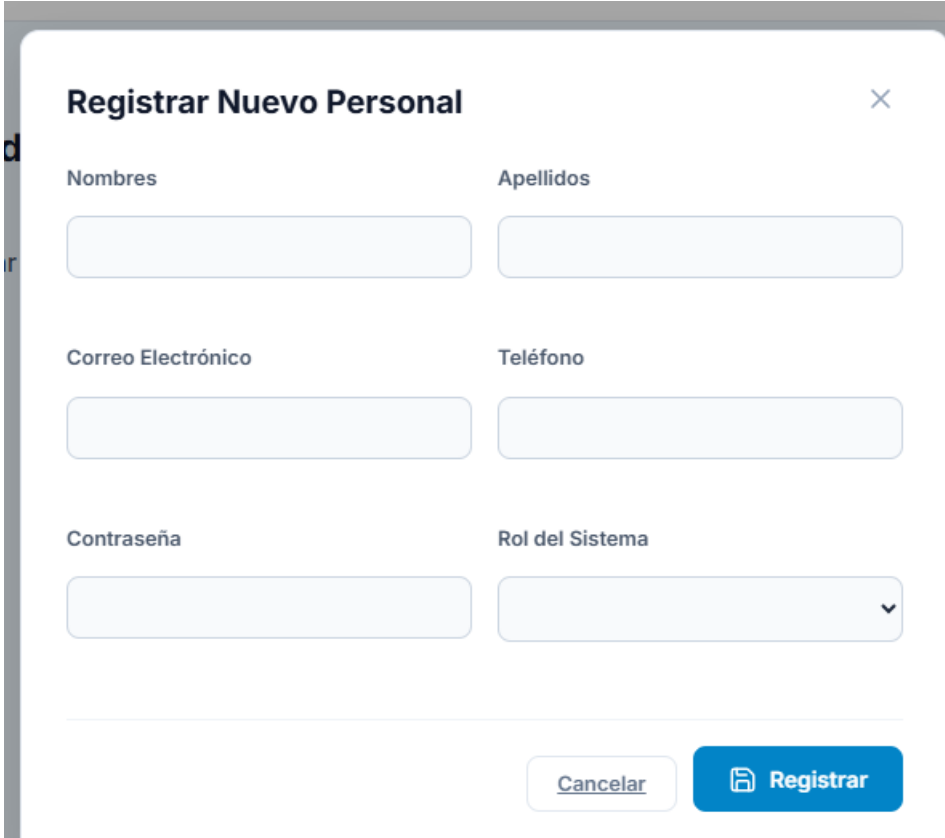
CU-04:Gestion de personal



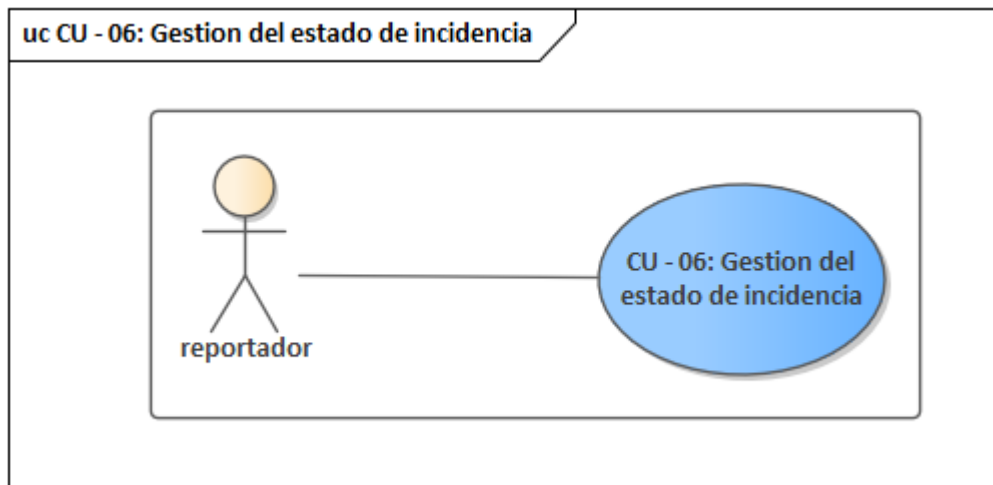
Campo	Descripción
Caso de Uso	CU - 04: Gestión de usuarios
Propósito	Administrar las cuentas de acceso para Encargados de Mantenimiento
Descripción	se deberá poder administrar las cuentas que tengan acceso al software
Actores	Administrador

Actor Iniciador	Administrador
Precondición	se tiene que iniciar sesión

Proceso	Registro 1. Se loguea 2. Se va a la vista de usuarios 3. Se le da al botón de registro 4. Se rellena el formulario (nombres, apellidos, correo, teléfono, contraseña, rol,) 5. El sistema ejecuta la Cadena de Responsabilidad validando secuencialmente: nombres → correo institucional → contraseña → rol 6. Si todas las validaciones pasan, se registra al usuario 7. Si alguna validación falla, la cadena se detiene y muestra el error específico Actualización 1. Se loguea 2. Se va a la vista de usuarios 3. Se le da al botón de actualizar al usuario correspondiente 4. Se rellena el formulario 5. Se actualiza al usuario escogido Eliminar 1. Se loguea 2. Se va a la vista de usuarios 3. Se le da al botón de eliminar al usuario correspondiente 4. Se elimina al usuario escogido (eliminación)
Postcondición	Se avisara si el proceso fue exitoso o no al usuario del registro, actualizacion y eliminacion

Campo	Descripción
Excepciones	los formularios de registro y actualizacion van a verificar los datos, osea que no se mande datos incorrectos
prototipo	

CU-06:Gestion del estado de Incidencia

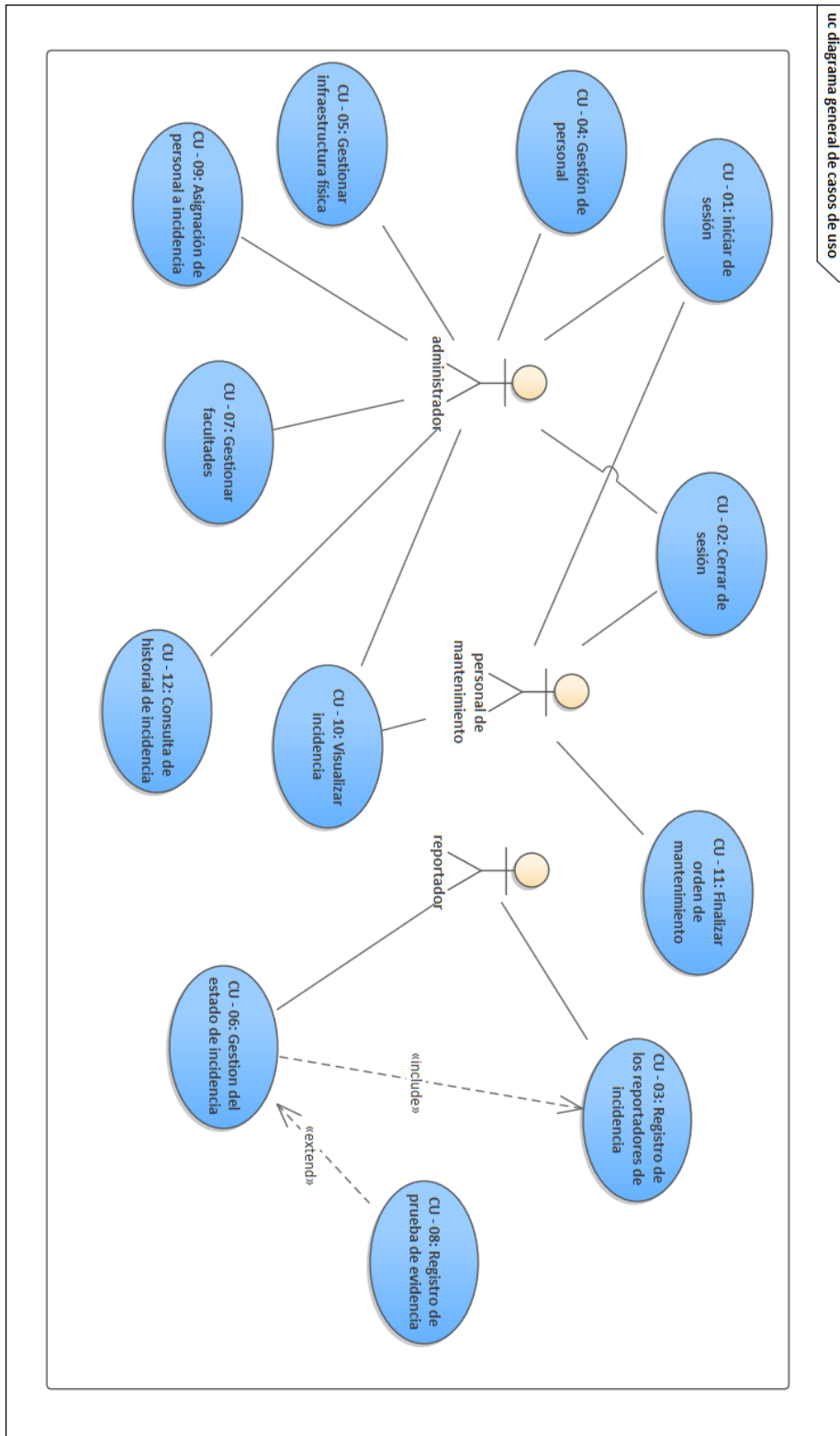


Campo	Descripción
Caso de Uso	CU-06:Gestion del estado de Incidencia
Propósito	permitir que los usuarios reporten incidencias y controlar el ciclo de vida de cada incidencia mediante transiciones de estado validadas
Descripción	los problemas encontrados por estudiantes, docentes u otros se registran para tener constancia de estos. Una vez registrada, la incidencia pasa por un ciclo de vida controlado por el Patrón Estado (State): Pendiente → Asignado → En Proceso → Finalizado. Cada estado permite o bloquea acciones específicas.
Actores	Reportador , Administrador, Personal de Mantenimiento
Actor Iniciador	Reportador

Campo	Descripción
Precondición	El actor debe estar autenticado en el sistema.
Proceso	1. Registro (Reportador) 1. El Reportador inicia sesión en la plataforma. 2. Ingresa al formulario de registro de problemas. 3. Rellena los datos básicos: título, descripción, ubicación (bloque/aula). 4. El sistema crea la incidencia e inicializa su estado por defecto como Pendiente (EstadoPendiente). 2. Asignación (Administrador) 1. El Administrador visualiza las incidencias en estado Pendiente. 2. Selecciona una incidencia y le asigna un técnico de la lista de Personal de Mantenimiento. 3. El sistema delega la acción al estado actual (EstadoPendiente::asignarPersonal()). 4. Se registra el técnico en la base de datos y la incidencia transiciona automáticamente al estado Asignado (EstadoAsignado). 3. Ejecución y Registro de Avances (Personal de Mantenimiento) 1. El Personal de Mantenimiento asignado ve la incidencia en su bandeja de trabajo. 2. Al iniciar la reparación o al registrar el primer avance, sube evidencias (fotos/notas/insumos). 3. El sistema delega la acción al estado actual (EstadoAsignado::agregarEvidencia()). 4. Al recibir la primera evidencia, el sistema cambia automáticamente el estado a En Proceso (EstadoEnProceso). 5. El técnico puede seguir añadiendo más evidencias de avance mientras la incidencia se mantenga En Proceso (sin alterar su estado). 4. Cierre de la Incidencia (Personal de Mantenimiento / Administrador)

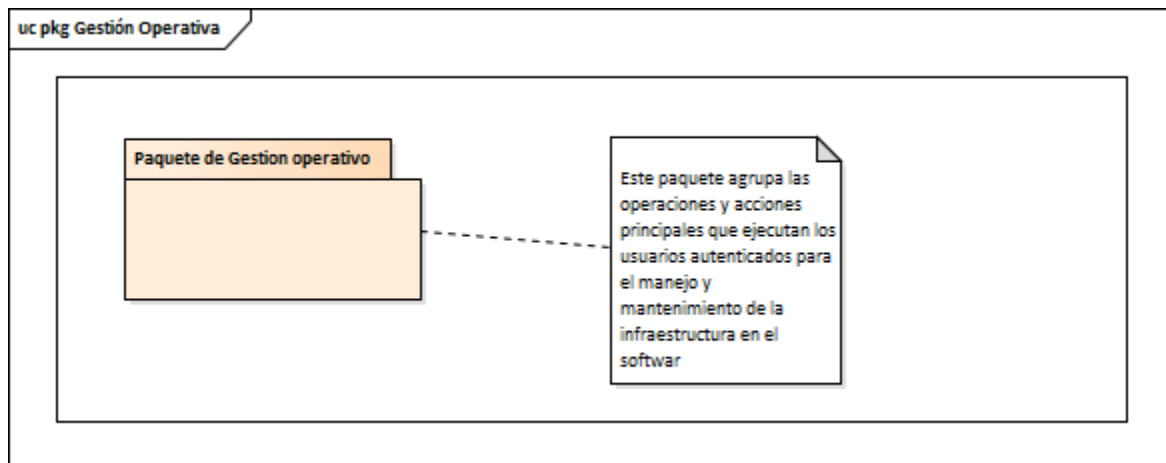
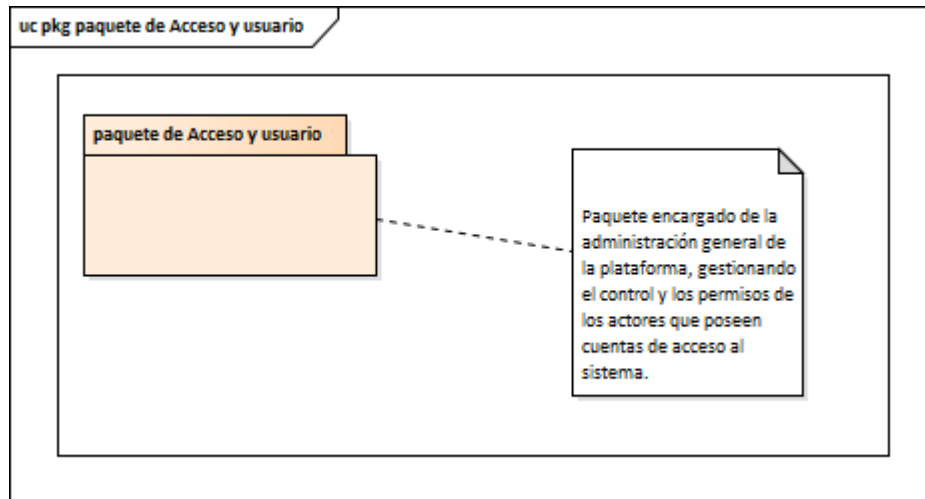
	1. Una vez reparado el problema, el Personal de Mantenimiento o el Administrador marcan la incidencia como completada. 2. El sistema delega la acción al estado actual (EstadoEnProceso::finalizarIncidencia()). 3. El estado cambia a Finalizado (EstadoFinalizado). 4. A partir de este momento, el EstadoFinalizado bloquea cualquier acción posterior (no se pueden cambiar estados ni añadir más evidencias).
Postcondición	• se actualizará el estado de la incidencia en la base de datos en cada transición. • Si un actor intenta realizar una acción no permitida para el estado actual (por ejemplo, que el técnico intente finalizar una orden que el Administrador no ha asignado todavía, o intentar modificar un reporte ya Finalizado), el Patrón Estado interceptará la petición en el backend y retornará una excepción controlada impidiendo la operación.
Excepciones	los formularios validaran los datos correctamente
prototipo	

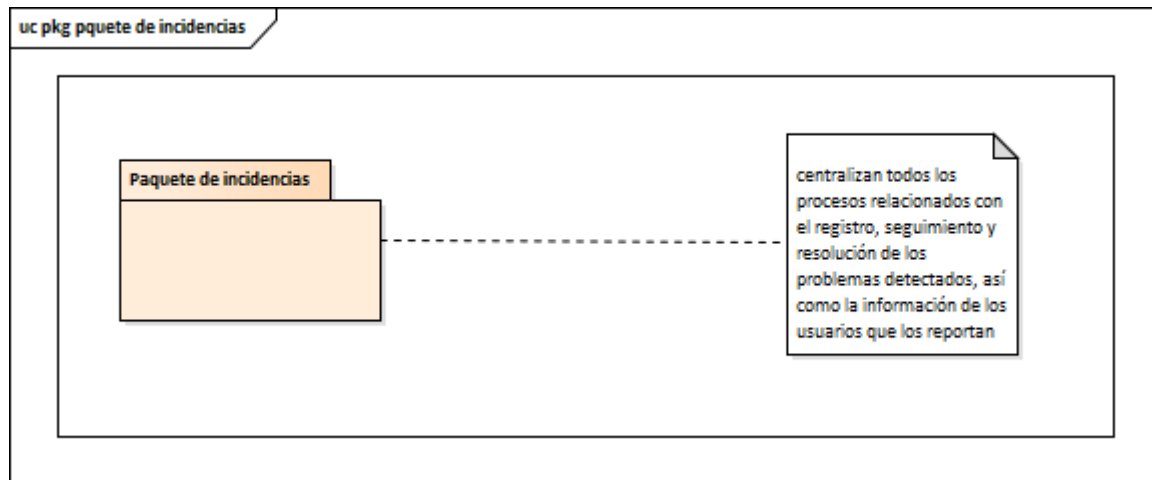
1.7. Diagrama general de Casos de Uso



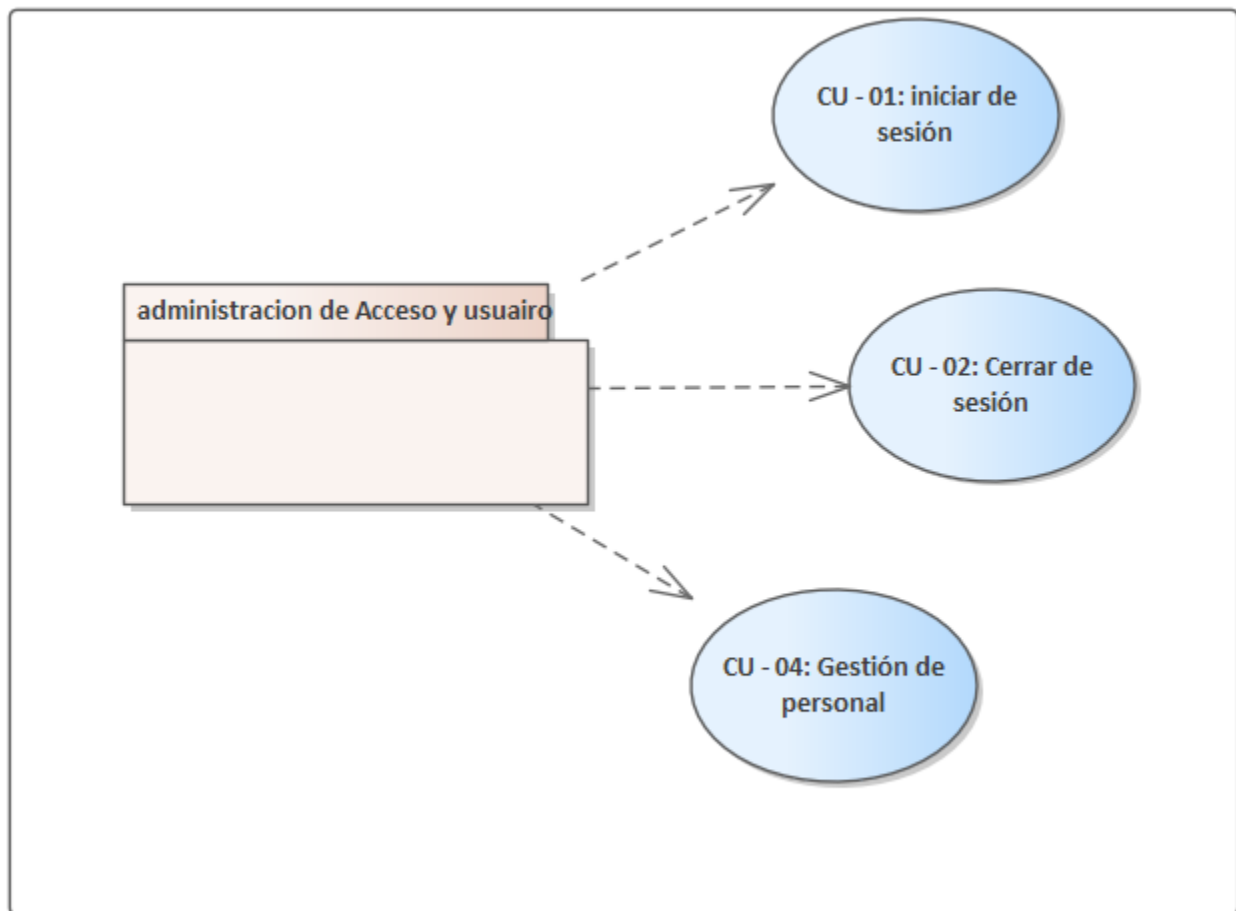
CAPITULO 2: ANALISIS

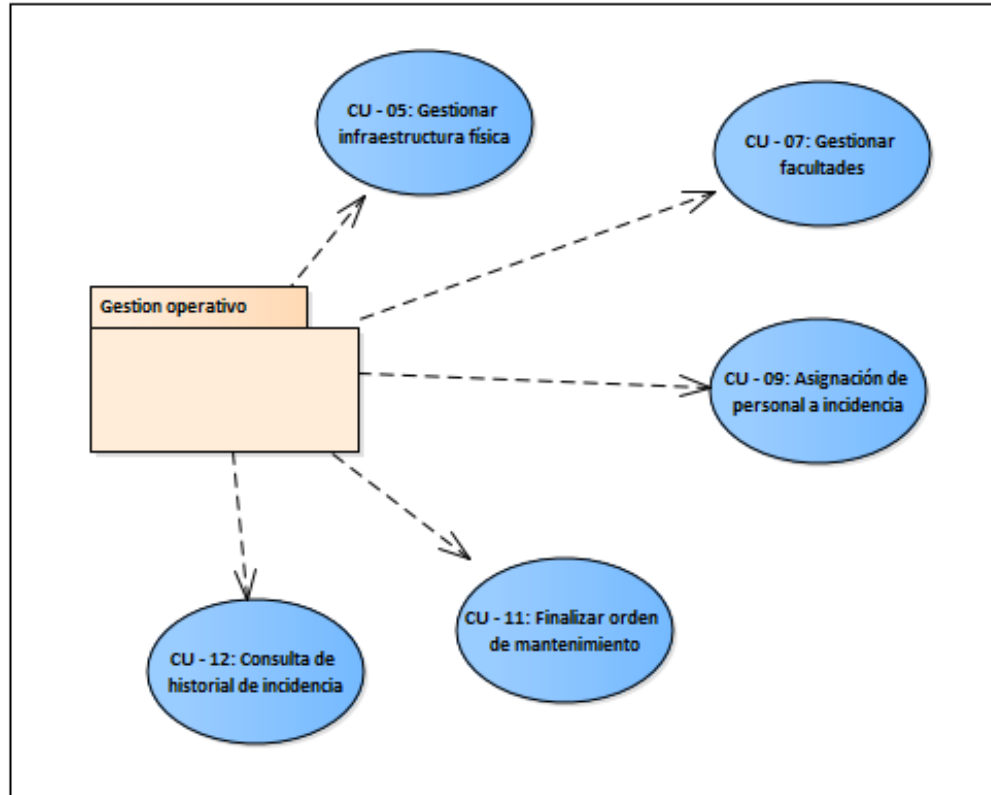
2.1. identificación de Paquetes

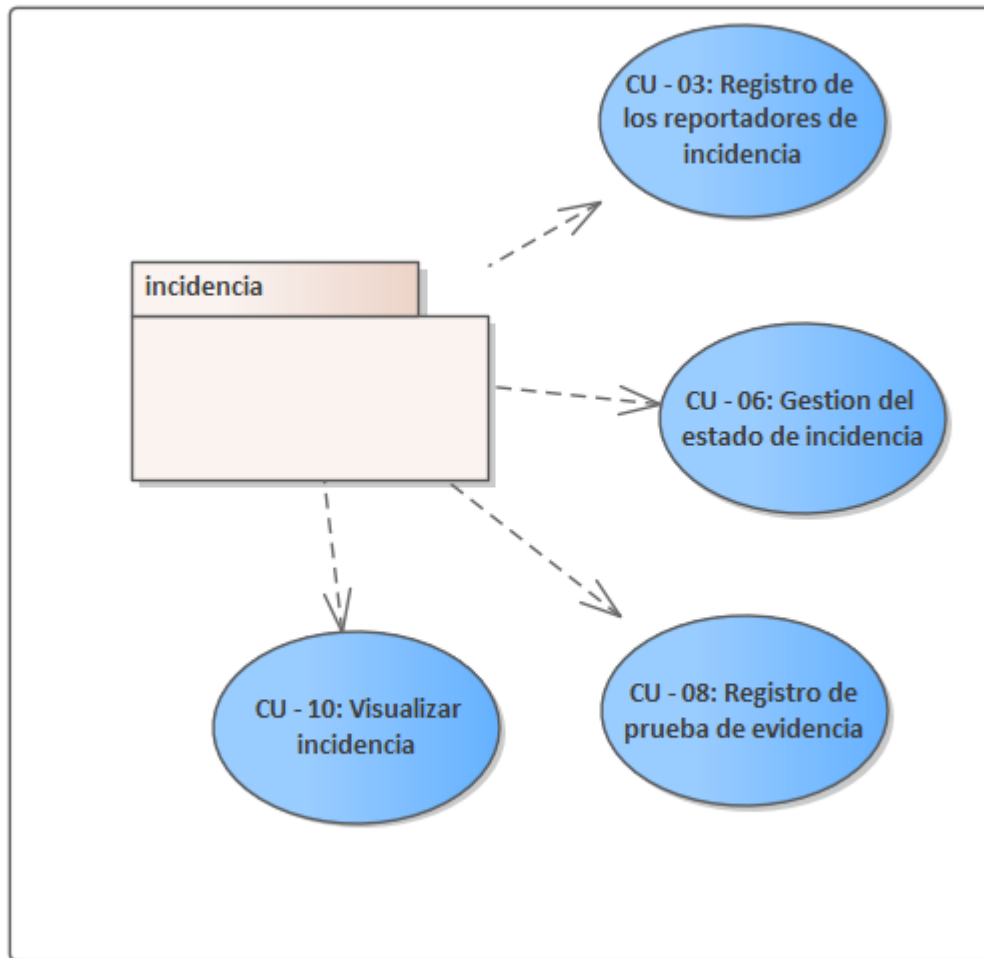




2.2. relacionar paquetes con Casos de Uso



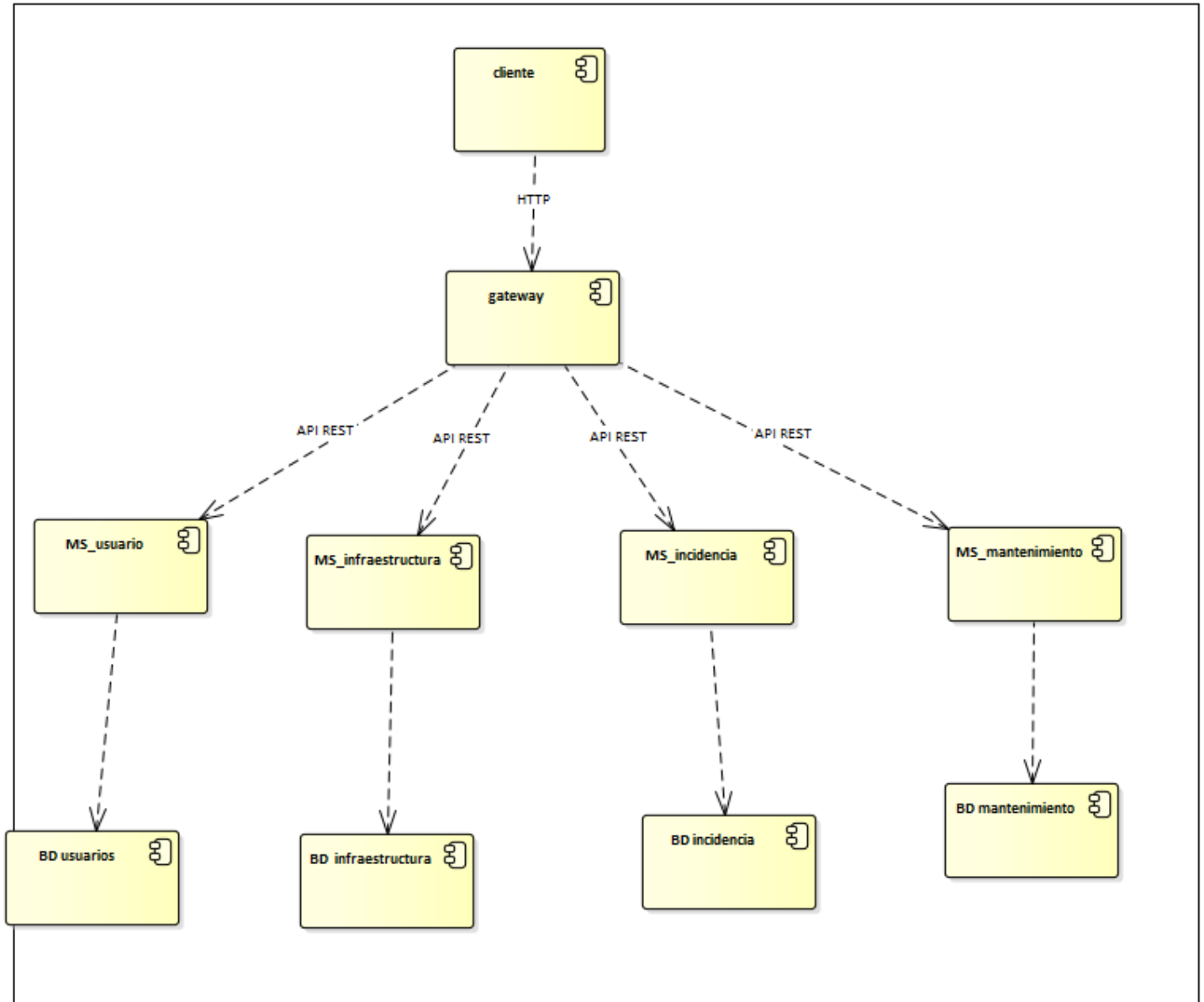




CAPTULO 3: DISEÑO

3.1. Diseño de la Arquitectura

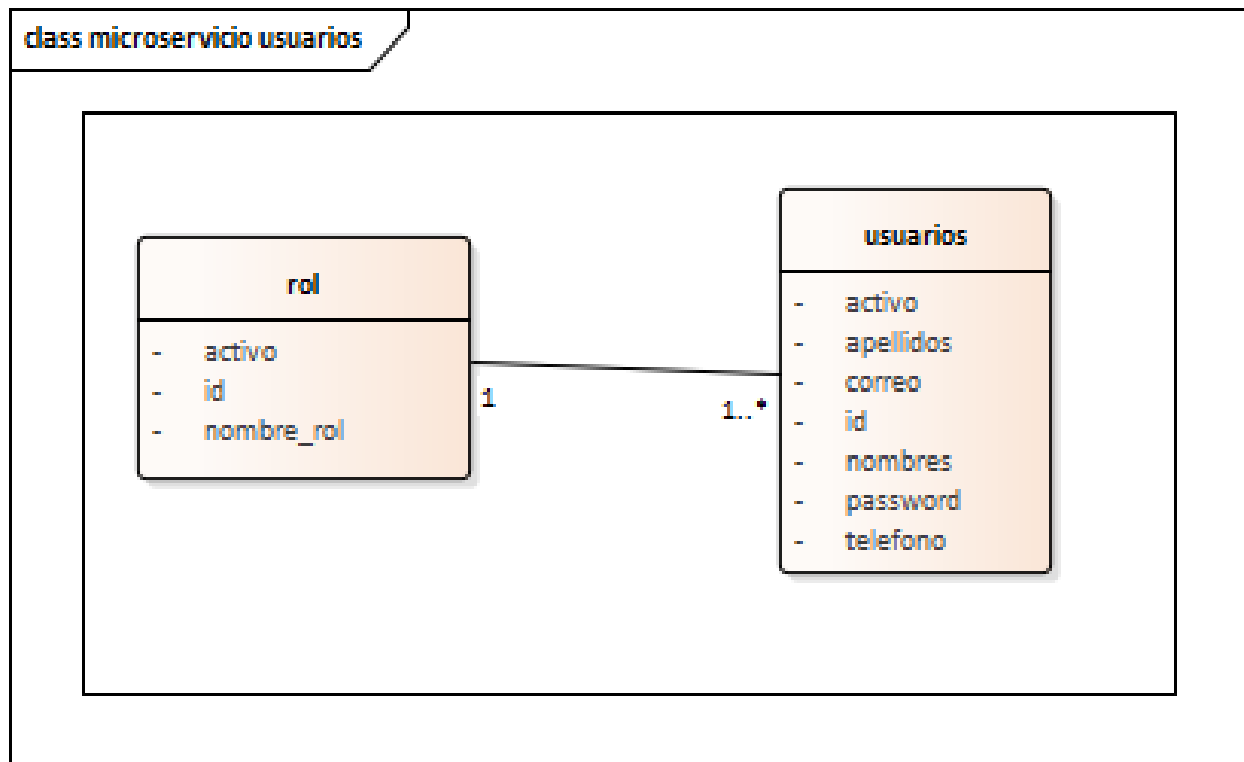
cmp diseño de la arquitectura



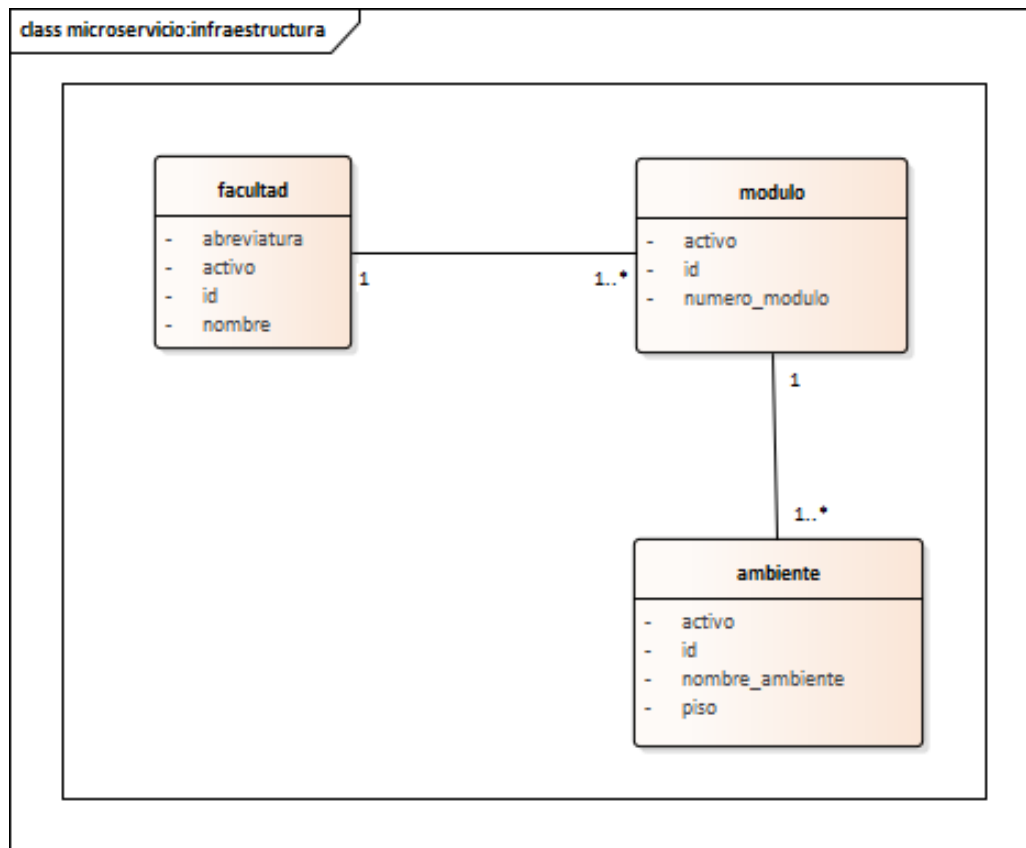
3.2. Diseño de la Base de Datos

3.2.1. Diseño conceptual

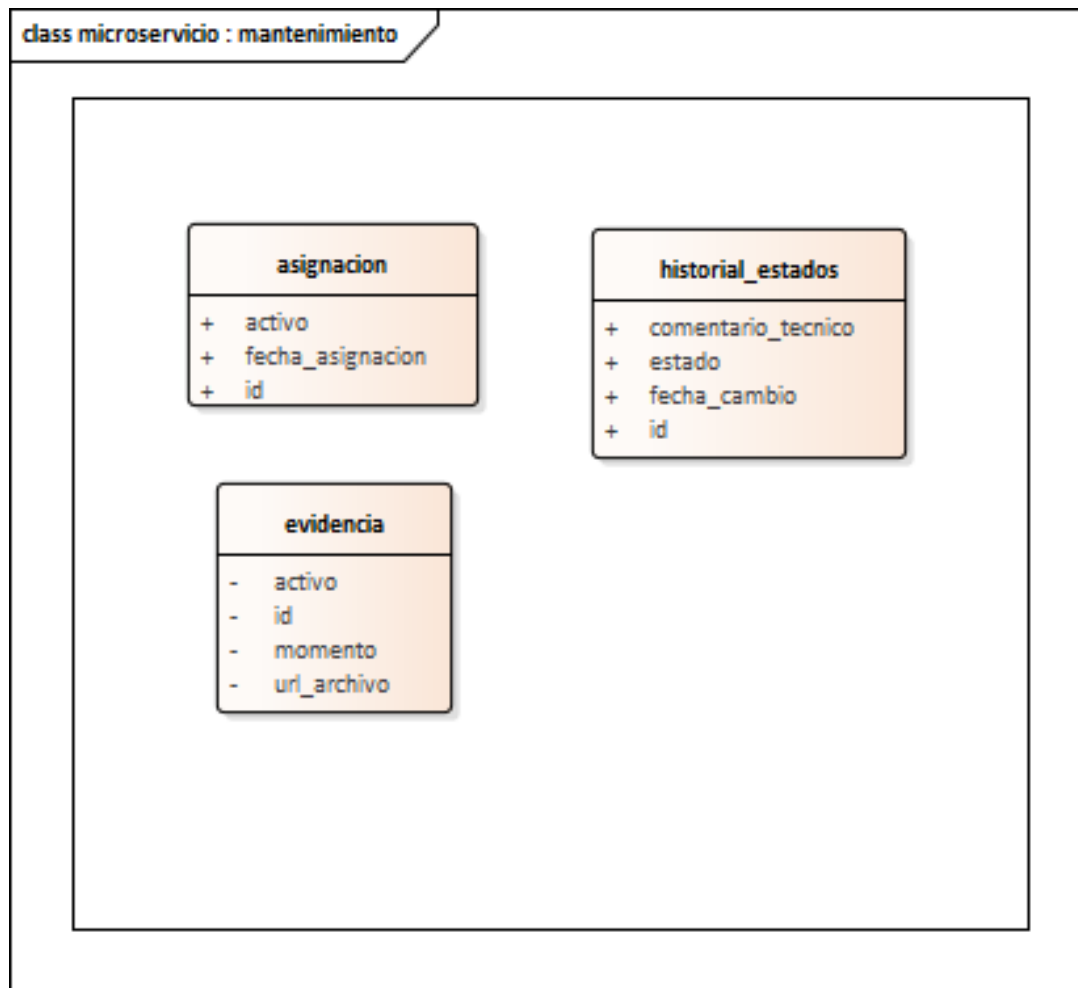
Usuarios



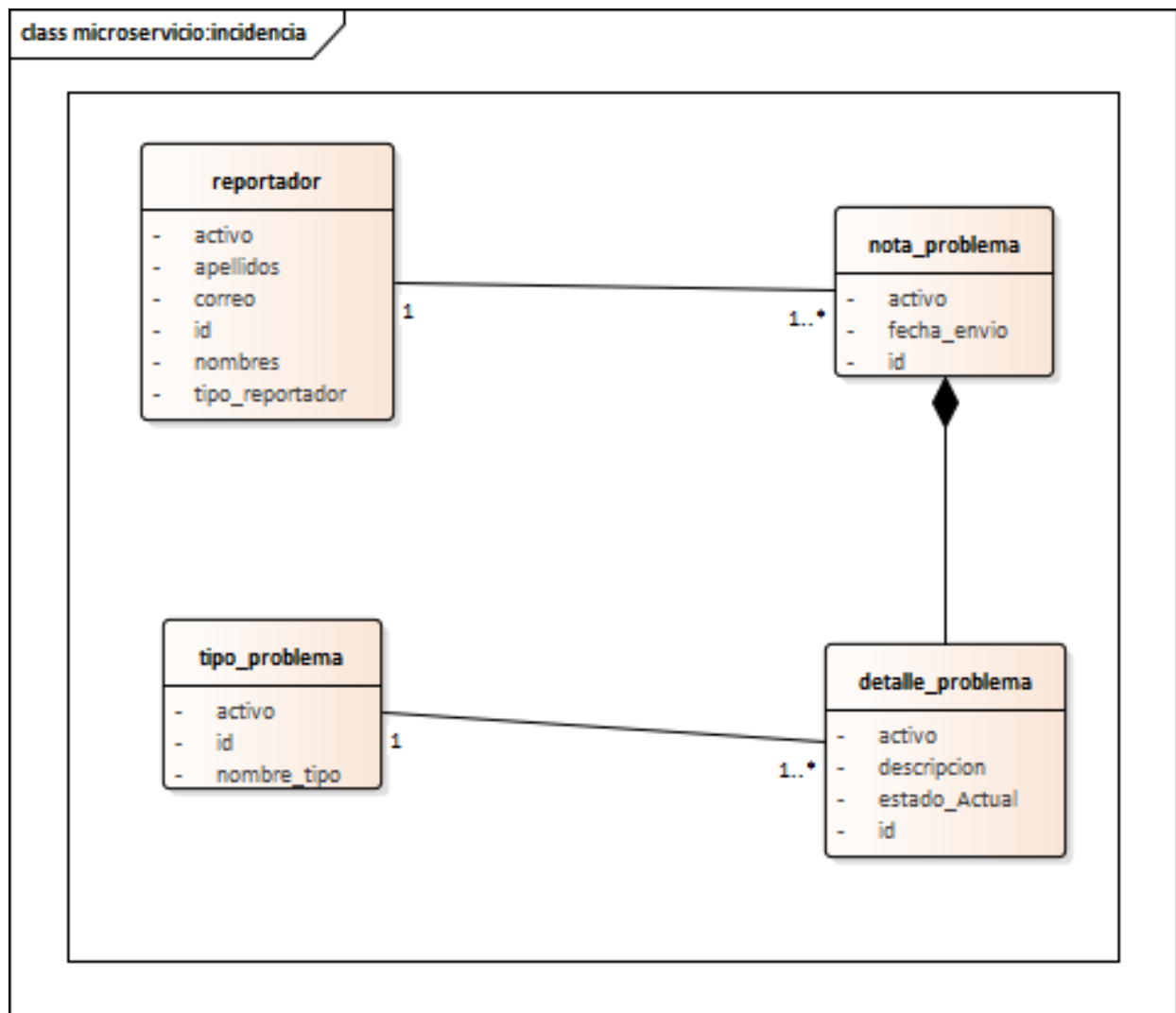
Infraestructura



Mantenimiento



Incidencia



3.2.2. Diseño Logico

Usuarios

rol

PK	
id	nombre_rol

usuarios

PK						FK		
id	nombres	apellidos	telefono	correo	password	rol_id	facultad_id	activo

Infraestructura

facultad

PK			
id	nombre	abreviatura	activo

ambiente

PK			FK	
id	nombre_ambiente	piso	modulo_id	activo

modulo

PK		FK	
id	nombre_modulo	facultad_id	activo

MS:Incidencia

reportador

PK					
id	nombres	apellidos	correo	tipo_reportador	activo

tipo_incidencia

PK		
id	nombre_tipo	activo

nota_problemas

PK		FK	
id	fecha_envio	reportador_id	activo

detalle_problema

PK				FK	FK	
id	descripcion	estado_actual	ambiente_id	nota_id	tipo_incidencia_id	activo

MS_Mantenimiento

asignacion

PK				
id	fecha_asignacion	detalle_problema_id	usuario_id	activo

historial_Estados

PK					FK	
id	tipo	estado	fecha_cambio	comentario_tecnico	asignacion_id	activo

evidencia

PK			FK	
id	url_archivo	momento	asignacion_id	activo

3.2.3. Diseño Fisico

Usuarios

```
CREATE TABLE IF NOT EXISTS "rol" (  
    "id" SERIAL PRIMARY KEY,  
    "nombre_rol" VARCHAR(50) NOT NULL,  
    "activo" BOOLEAN DEFAULT TRUE  
);
```

```
CREATE TABLE IF NOT EXISTS "usuarios" (  
    "id" SERIAL PRIMARY KEY,  
    "nombres" VARCHAR(100) NOT NULL,  
    "apellidos" VARCHAR(100) NOT NULL,  
    "telefono" VARCHAR(20),  
    "correo" VARCHAR(100) UNIQUE NOT NULL,  
    "password" VARCHAR(255) NOT NULL,  
    "rol_id" INTEGER REFERENCES "rol"("id") ON DELETE SET NULL,  
    "facultad_id" INTEGER,  
    "activo" BOOLEAN DEFAULT TRUE  
);
```

Infraestructura

```
CREATE TABLE IF NOT EXISTS "facultad" (  
    "id" SERIAL PRIMARY KEY,  
    "nombre" VARCHAR(150) NOT NULL,  
    "abreviatura" VARCHAR(10),
```

```
"activo" BOOLEAN DEFAULT TRUE  
);
```

```
CREATE TABLE IF NOT EXISTS "modulo" (  
    "id" SERIAL PRIMARY KEY,  
    "numero_modulo" VARCHAR(20) NOT NULL,  
    "facultad_id" INTEGER REFERENCES "facultad"("id") ON DELETE CASCADE,  
    "activo" BOOLEAN DEFAULT TRUE  
);
```

```
CREATE TABLE IF NOT EXISTS "ambiente" (  
    "id" SERIAL PRIMARY KEY,  
    "nombre_ambiente" VARCHAR(100) NOT NULL,  
    "piso" INTEGER,  
    "modulo_id" INTEGER REFERENCES "modulo"("id") ON DELETE CASCADE,  
    "activo" BOOLEAN DEFAULT TRUE  
);
```

Incidencia

```
CREATE TABLE IF NOT EXISTS "reportador" (  
    "id" SERIAL PRIMARY KEY,  
    "nombres" VARCHAR(100) NOT NULL,  
    "apellidos" VARCHAR(100) NOT NULL,  
    "correo" VARCHAR(100) NOT NULL,  
    "tipo_reportador" VARCHAR(50) NOT NULL, -- Estudiante, Docente, Administrativo  
    "activo" BOOLEAN DEFAULT TRUE  
);
```

```
CREATE TABLE IF NOT EXISTS "tipo_incidencia" (
    "id" SERIAL PRIMARY KEY,
    "nombre_tipo" VARCHAR(100) NOT NULL, -- Eléctrico, Mobiliario, Paredes, etc.
    "activo" BOOLEAN DEFAULT TRUE
);
```

```
CREATE TABLE IF NOT EXISTS "nota_problemas" (
    "id" SERIAL PRIMARY KEY,
    "fecha_envio" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "reportador_id" INTEGER REFERENCES "reportador"("id") ON DELETE SET NULL,
    "activo" BOOLEAN DEFAULT TRUE
);
```

```
CREATE TABLE IF NOT EXISTS "detalle_problema" (
    "id" SERIAL PRIMARY KEY,
    "descripcion" TEXT NOT NULL,
    "estado_actual" VARCHAR(50) NOT NULL DEFAULT 'Pendiente',
    "ambiente_id" INTEGER, -- Atributo de referencia (Ubicación)
    "nota_id" INTEGER REFERENCES "nota_problemas"("id") ON DELETE CASCADE,
    "tipo_incidencia_id" INTEGER REFERENCES "tipo_incidencia"("id") ON DELETE SET NULL,
    "activo" BOOLEAN DEFAULT TRUE
);
```

Mantenimiento

```
CREATE TABLE IF NOT EXISTS "asignacion" (
    "id" SERIAL PRIMARY KEY,
    "fecha_asignacion" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "detalle_problema_id" INTEGER, -- Atributo de referencia (Reporte de Problemas)
```

```

"usuario_id" INTEGER,      -- Atributo de referencia (Usuarios - Técnico)

"activo" BOOLEAN DEFAULT TRUE

);

CREATE TABLE IF NOT EXISTS "historial_estados" (

    "id" SERIAL PRIMARY KEY,

    "tipo" VARCHAR(50) NOT NULL, -- Ej: Cambio de Estado, Comentario

    "estado" VARCHAR(50) NOT NULL, -- Ej: Asignado, En Proceso, Resuelto

    "fecha_cambio" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    "comentario_tecnico" TEXT,

    "asignacion_id" INTEGER REFERENCES "asignacion"("id") ON DELETE CASCADE,

    "activo" BOOLEAN DEFAULT TRUE

);

CREATE TABLE IF NOT EXISTS "evidencia" (

    "id" SERIAL PRIMARY KEY,

    "url_archivo" VARCHAR(255) NOT NULL,

    "momento" VARCHAR(20) NOT NULL, -- Antes, Despues

    "asignacion_id" INTEGER REFERENCES "asignacion"("id") ON DELETE CASCADE,

    "activo" BOOLEAN DEFAULT TRUE

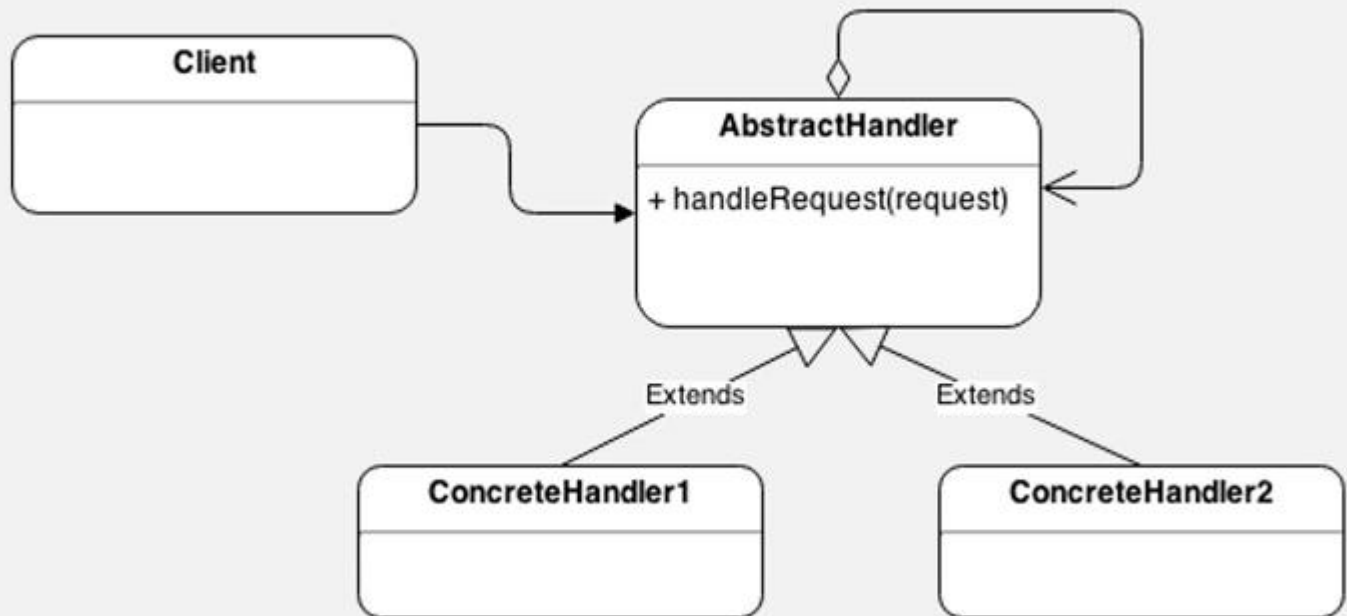
);

```

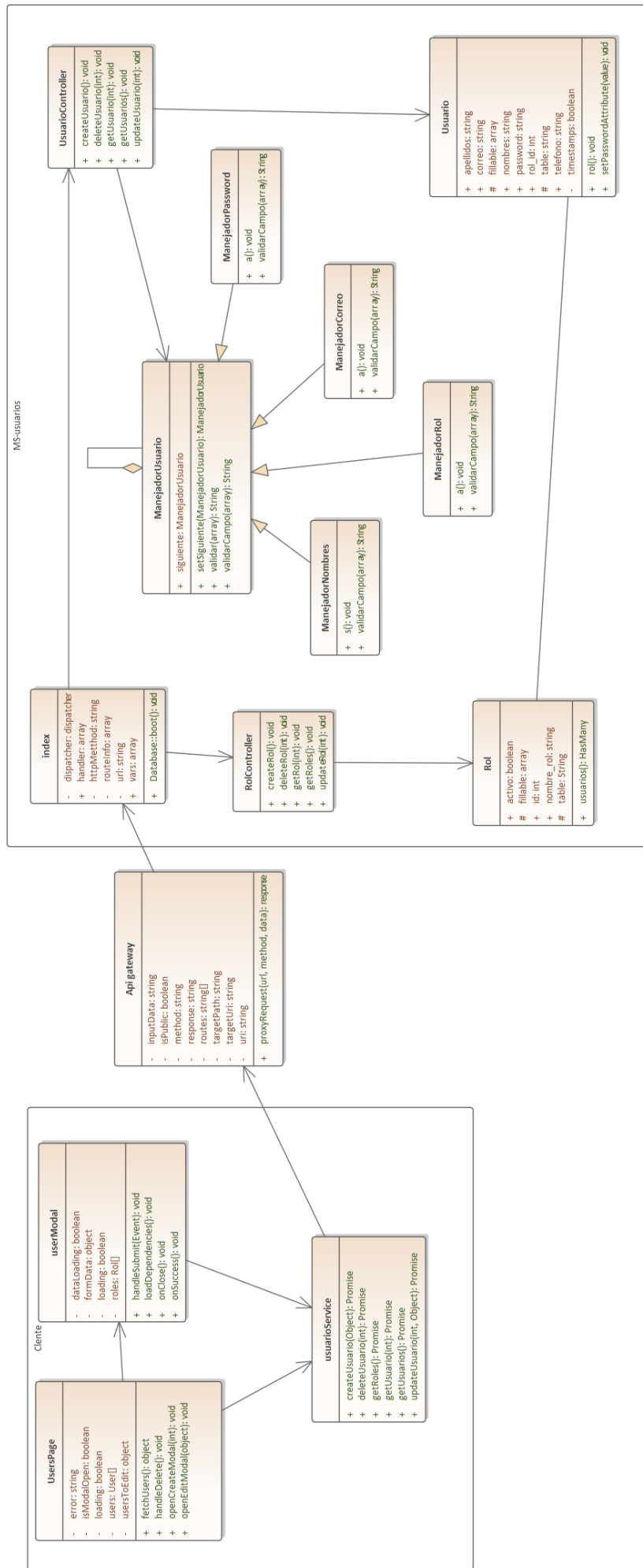
3.3. Diseño de detalle procedimental

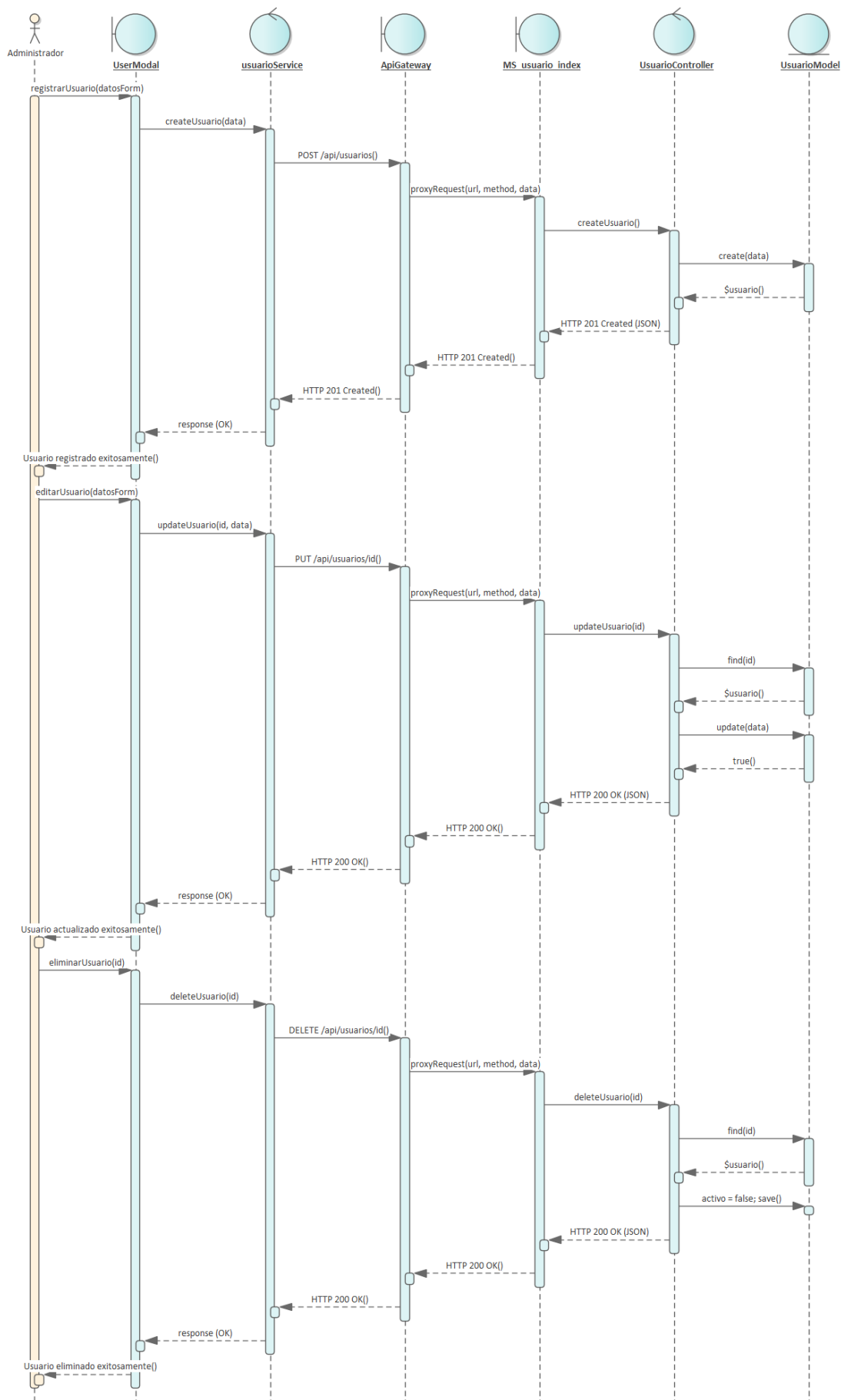
Patron de cadena de responsabilidad

Chain Of Responsibility pattern – Class diagram

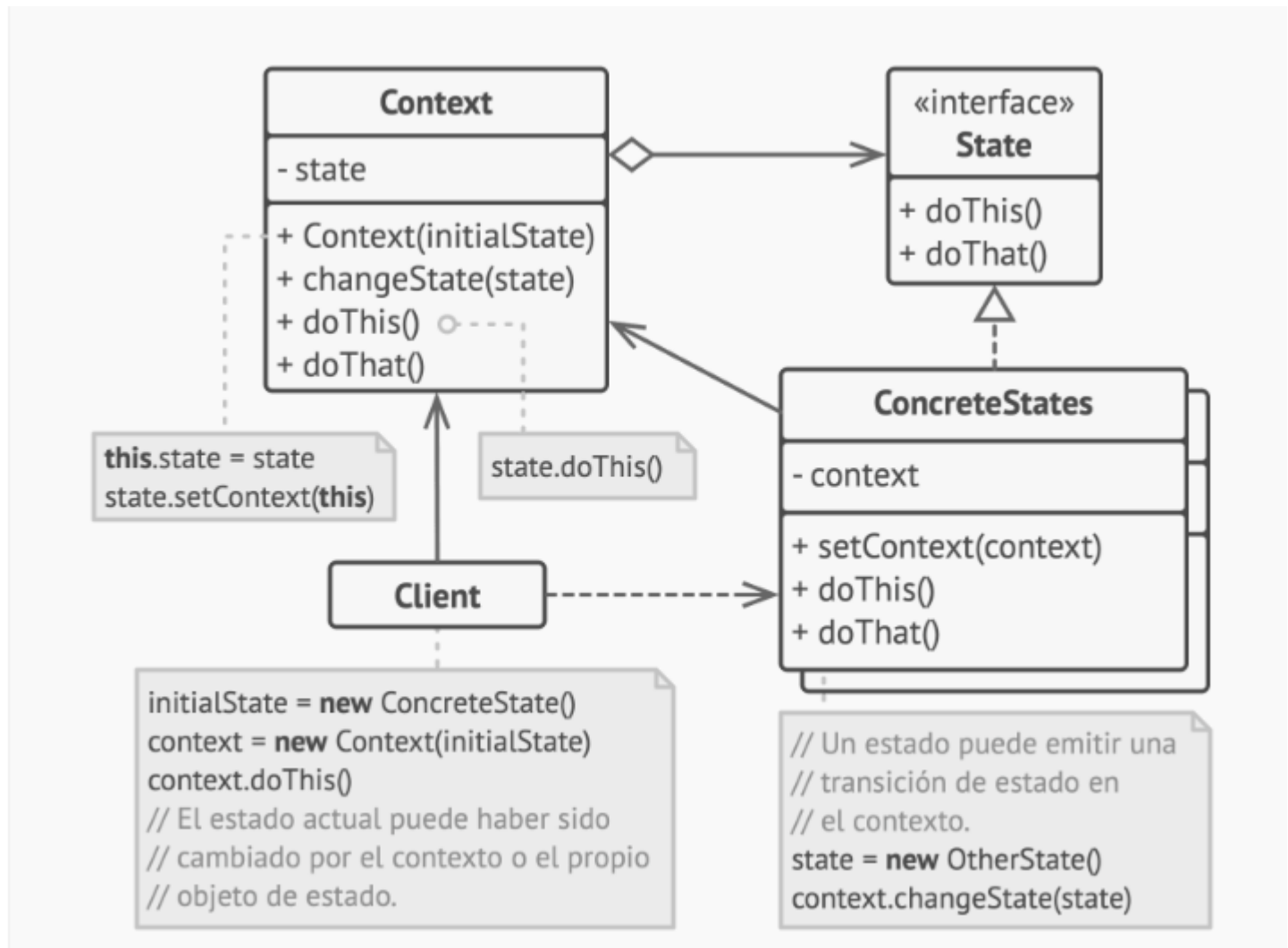


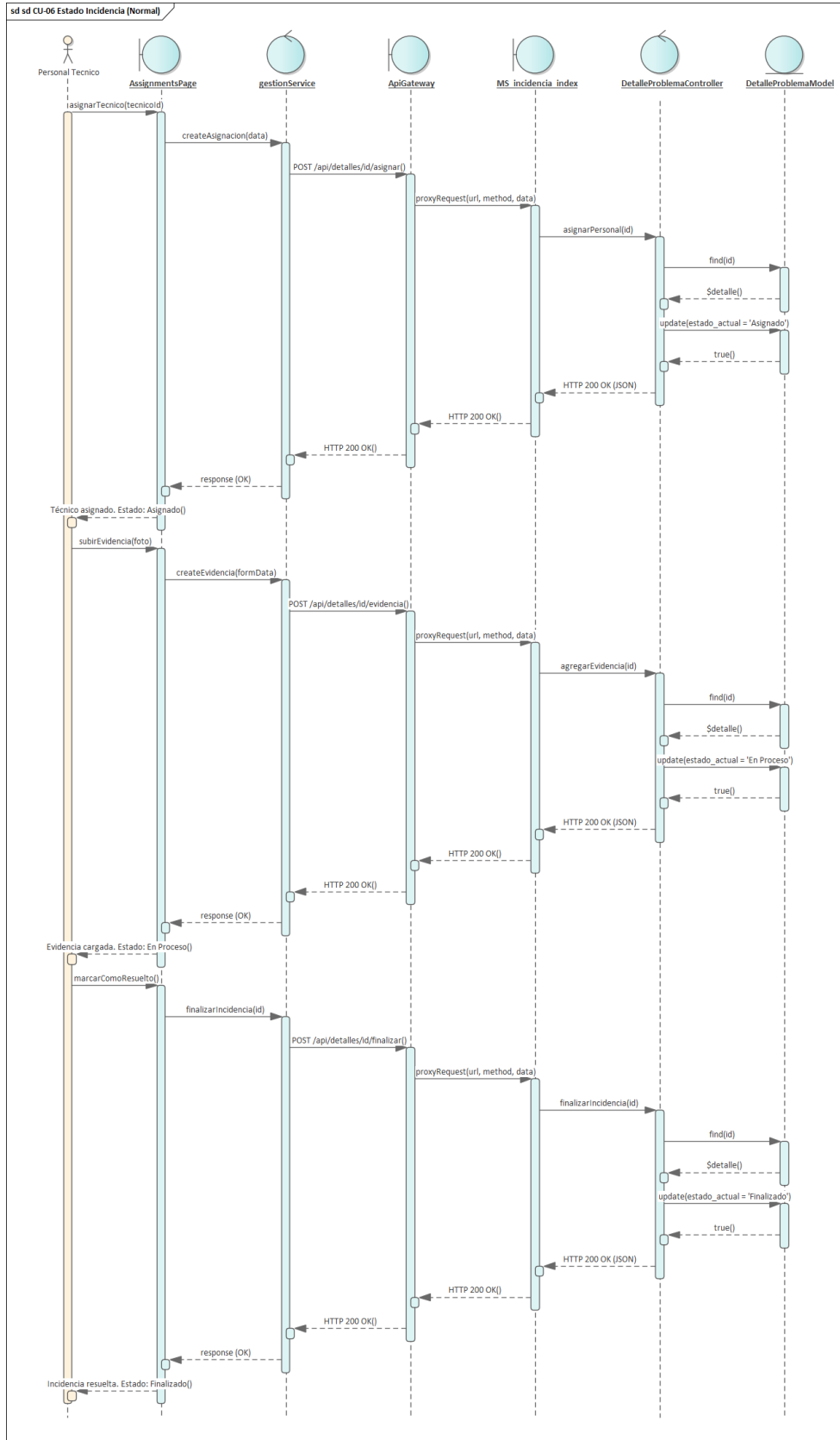
CU-04:Gestion de usuarios





Patron de estado (estate)





Código

<https://github.com/Jorgito-cc/arqui2.git>